

Solvers for dense and sparse quadratic problems

Alexandre Gramfort

Master 2 Data Science, Univ. Paris Saclay
Optimisation for Data Science

Unconstrained optimization problem

Definition (Unconstrained optimization problem ($\mathcal{P}$))

$$\min_{x \in \mathbb{R}^n} f(x)$$

- where $f : \mathbb{R}^n \rightarrow \mathbb{R} \cup \{+\infty\}$ is the **objective function**

Quadratic functions

Definition (Quadratic form)

A quadratic form reads

$$q(x) = \frac{1}{2}x^\top Ax - b^\top x + c$$

where $x \in \mathbb{R}^n$, $A \in \mathbb{R}^{n \times n}$, $b \in \mathbb{R}^n$ and $c \in \mathbb{R}$.

→ What equation do stationary points satisfy?

→ What condition on A do we need to guarantee the existence and uniqueness of x^* ?

→ Show that minimizing q boils down to solving a linear system.

Taylor at order 2

Assuming f is twice differentiable, the Taylor expansion at order 2 of f at x reads:

$$\forall h \in \mathbb{R}^n, f(x+h) = f(x) + \nabla f(x)^\top h + \frac{1}{2} h^\top \nabla^2 f(x) h + o(\|h\|^2)$$

- $\nabla f(x) \in \mathbb{R}^n$ is the gradient.
- $\nabla^2 f(x) \in \mathbb{R}^{n \times n}$ the Hessian matrix.

Remark: It gives a local quadratic approximation

→ Show that if $\nabla^2 f(x) = L I$ then minimizing the quadratic approximation leads to gradient descent. With what step size?

Ridge regression

We consider problems with n samples, observations, and p features, variables. WARNING: Using standard ML notations (X, y)

Definition (Ridge regression)

Let $y \in \mathbb{R}^n$ the n targets to predict and $(x^i)_i$ the n samples in $\mathbb{R}^p$. Ridge regression consists in solving the following problem

$$\min_{w, b} \frac{1}{2} \|y - Xw - b\mathbf{1}_n\|^2 + \frac{\lambda}{2} \|w\|^2, \lambda > 0$$

where $w \in \mathbb{R}^p$ is called the weights vector, $b \in \mathbb{R}$ is the intercept (a.k.a. bias) and the i th row of X is x^i .

Remark: We have an optimization problem in dimension $p + 1$

Remark: Note that the intercept is not penalized with λ .

Taking care of the intercept

Exercise

Let

$$\hat{w}, \hat{b} = \arg \min_{w, b} \frac{1}{2} \|y - Xw - b\mathbf{1}_n\|^2 + \frac{\lambda}{2} \|w\|^2, \lambda > 0$$

$\bar{y} \in \mathbb{R}$ the mean of y and $\bar{X} \in \mathbb{R}^p$ the mean of each column of X .
→ Show that $\hat{b} = -\bar{X}^\top \hat{w} + \bar{y}$.

Taking care of the intercept

Exercise

Let

$$\hat{w}, \hat{b} = \arg \min_{w, b} \frac{1}{2} \|y - Xw - b\mathbf{1}_n\|^2 + \frac{\lambda}{2} \|w\|^2, \lambda > 0$$

$\bar{y} \in \mathbb{R}$ the mean of y and $\bar{X} \in \mathbb{R}^p$ the mean of each column of X .
→ Show that $\hat{b} = -\bar{X}^\top \hat{w} + \bar{y}$.

Ways to deal with the intercept:

- Option 1 (dense case): Center the target y and each column feature and solve:

$$\min_{w \in \mathbb{R}^p} \frac{1}{2} \|y - Xw\|^2 + \frac{\lambda}{2} \|w\|^2$$

- Option 2 (sparse case): Add a column of 1 to X and try not to penalize it (too much).

Ridge regression

We consider:

$$\min_{w \in \mathbb{R}^p} \frac{1}{2} \|y - Xw\|^2 + \frac{\lambda}{2} \|w\|^2$$

Exercise

- *Show that ridge regression boils down to the minimization of a quadratic form.*
- *Propose a closed form solution.*
- *Show that the solution is obtained by solving a linear system.*
- *Is the objective function strongly convex?*
- *Assuming $n < p$ what is the value of the constant of strong convexity?*

→ cf. notebook

Singular value decomposition (SVD)

- SVD is a factorization of a matrix (real here)
- $M = U\Sigma V^T$ where $M \in \mathbb{R}^{n \times p}$, $U \in \mathbb{R}^{n \times n}$, $\Sigma \in \mathbb{R}^{n \times p}$, $V \in \mathbb{R}^{p \times p}$
- $U^T U = U U^T = I_n$ (orthogonal matrix)
- $V^T V = V V^T = I_p$ (orthogonal matrix)
- Σ diagonal matrix
- $\Sigma_{i,i}$ are called the singular values
- U are left-singular vectors
- V are right-singular vectors

Singular value decomposition (SVD)

- SVD is a factorization of a matrix (real here)
- U contains the eigenvectors of $MM^\top$ associated to the eigenvalues $\Sigma_{i,i}^2$ for $1 \leq i \leq n$.
- V contains the eigenvectors of $M^\top M$ associated to the eigenvalues $\Sigma_{i,i}^2$ for $1 \leq i \leq p$.
- we assume here $\Sigma_{i,i} = 0$ for $\min(n, p) < i \leq \max(n, p)$
- SVD is particularly useful to find the rank, null-space, image and pseudo-inverse of a matrix

Matrix inversion lemma

Proposition (Matrix inversion lemma)

also known as “Woodbury matrix identity” states that:

$$(A + UCV)^{-1} = A^{-1} - A^{-1}U(C^{-1} + VA^{-1}U)^{-1}VA^{-1},$$

where $A \in \mathbb{R}^{n \times n}$, $U \in \mathbb{R}^{n \times k}$, $C \in \mathbb{R}^{k \times k}$, $V \in \mathbb{R}^{k \times n}$.

Matrix inversion lemma

Proposition (Matrix inversion lemma)

also known as “Woodbury matrix identity” states that:

$$(A + UCV)^{-1} = A^{-1} - A^{-1}U(C^{-1} + VA^{-1}U)^{-1}VA^{-1},$$

where $A \in \mathbb{R}^{n \times n}$, $U \in \mathbb{R}^{n \times k}$, $C \in \mathbb{R}^{k \times k}$, $V \in \mathbb{R}^{k \times n}$.

Proof. Just check that $(A + UCV)$ times the RHS of the Woodbury identity gives the identity matrix:

$$\begin{aligned} & (A + UCV) \left[A^{-1} - A^{-1}U(C^{-1} + VA^{-1}U)^{-1}VA^{-1} \right] \\ &= I + UCVA^{-1} - (U + UCVA^{-1}U)(C^{-1} + VA^{-1}U)^{-1}VA^{-1} \\ &= I + UCVA^{-1} - UC(C^{-1} + VA^{-1}U)(C^{-1} + VA^{-1}U)^{-1}VA^{-1} \\ &= I + UCVA^{-1} - UCVA^{-1} = I \end{aligned}$$

Primal and dual implementation

We consider:

$$\min_{w \in \mathbb{R}^p} \frac{1}{2} \|y - Xw\|^2 + \frac{\lambda}{2} \|w\|^2$$

The solution is given by:

$$\hat{w} = (X^\top X + \lambda I_p)^{-1} X^\top y$$

Using matrix inversion lemma show that:

$$\hat{w} = X^\top (XX^\top + \lambda I_n)^{-1} y$$

This is a dual formulation and the matrix to invert is in $\mathbb{R}^{n \times n}$.

→ Using the SVD of X propose an implementation.

→ Can you use the SVD to confirm the primal-dual link?

→ What if X is sparse, n is $1e5$ and p is $1e6$?

Can you even store it?

Exercise

X is sparse with $n = 10^5$, $p = 10^6$ and 10 non-zeros per row, i.e., $\text{nnz}(X) = 10^6$. In `float64`, how much memory do you need for X , for $X^\top X$ and for $XX^\top$?

Can you even store it?

Exercise

X is sparse with $n = 10^5$, $p = 10^6$ and 10 non-zeros per row, i.e., $\text{nnz}(X) = 10^6$. In *float64*, how much memory do you need for X , for $X^\top X$ and for $XX^\top$?

X sparse (CSR)	nnz values + indices	≈ 12 MB
X dense	$n \times p$	800 GB
$X^\top X$ (primal)	$p \times p$	8 TB
$XX^\top$ (dual)	$n \times n$	80 GB

- The primal normal equations cannot even be *stored*, let alone solved.
- $X^\top X$ is dense in general, *even when X is sparse*.
- The dual is better but still out of reach, and SVD is not an option either.

Remark: X itself fits in 12 MB. The problem is not the data, it is the matrices we build from it.

Never form the matrix

We want to solve $Aw = X^\top y$ with $A = X^\top X + \lambda I_p$.

We never need A itself, only its action on a vector:

$$w \mapsto Aw = X^\top(Xw) + \lambda w$$

- Cost: $\approx 4 \operatorname{nnz}(X) = 4 \times 10^6$ flops, *i.e.*, milliseconds.
- Memory: a handful of vectors of size p .

→ We need a solver that only ever asks for products Aw .

→ Can you think of other settings where Aw is cheap but A is not available?

Remark: *e.g.*, Hessian-vector products obtained by automatic differentiation.

Conjugate gradient method: Solve $Ax = b$

The conjugate gradient method is an iterative method to solve linear systems with positive definite matrices ($A \succ 0$). **It only needs to know how to compute Ax** (operation can be implicit).

What we would like: at iteration k , move by the step p^k that minimizes f over *all* the information gathered so far, *i.e.*, the gradients at the previous iterates:

$$p^k = \arg \min_{p \in \text{span}(\nabla f(x^0), \dots, \nabla f(x^k))} f(x^k - p) \quad \text{and} \quad x^{k+1} = x^k - p^k .$$

Solving this subproblem naively would require storing the $k + 1$ gradients and solving a $(k + 1)$ -dimensional least-squares problem at every iteration.

→ **The conjugate gradient algorithm achieves exactly this** with a single vector of memory (d^{k-1}) and one product Ax per iteration. We write $p^k = \beta_k d^k$ and build the d^k recursively.

Conjugate gradient method: Solve $Ax = b$

Theorem (Convergence in n iterations)

The conjugate gradient algorithm finds the minimum of positive definite quadratic form q , in at most n iterations:

$$q(x) = \frac{1}{2}x^\top Ax - b^\top x + c ,$$

and therefore solves the linear system $Ax = b$ in at most n iterations.

Conjugate gradient method: Solve $Ax = b$

- Property
 - $\forall l < k, Ad^k \perp d^l$
 - i.e., vectors d^k and d^l are *conjugate* w.r.t. A
- Computation of the direction:
 - $d^k = g^k + \alpha_k d^{k-1}$ where $g^k = \nabla f(x^k)$ (we correct the gradient with a term that depends on previous iterations),

$$\alpha_k = -\frac{\langle g^k, Ad^{k-1} \rangle}{\langle Ad^{k-1}, d^{k-1} \rangle}$$

- Computation of optimal step size:

$$\beta_k = \frac{\langle g^k, d^k \rangle}{\langle Ad^k, d^k \rangle}$$

Conjugate gradient: Solve $Ax = b$

Require: $A \in \mathbb{R}^{n \times n}$ and $b \in \mathbb{R}^n$

```
1:  $x^0 \in \mathbb{R}^n$ ,  $g^0 = Ax^0 - b$ 
2: for  $k = 0$  to  $n - 1$  do
3:   if  $g^k = 0$  then
4:     return  $x^k$ 
5:   end if
6:   if  $k = 0$  then
7:      $d^k = g^0$ 
8:   else
9:      $\alpha_k = -\frac{\langle g^k, q^{k-1} \rangle}{\langle d^{k-1}, q^{k-1} \rangle}$ 
10:     $d^k = g^k + \alpha_k d^{k-1}$ 
11:  end if
12:   $q^k = Ad^k$  {the only product with  $A$ }
13:   $\beta_k = \frac{\langle g^k, d^k \rangle}{\langle d^k, q^k \rangle}$ 
14:   $x^{k+1} = x^k - \beta_k d^k$ 
15:   $g^{k+1} = g^k - \beta_k q^k$ 
16: end for
17: return  $x^n$ 
```

Remark: q^k is reused by β_k , by g^{k+1} and by α_{k+1} : one product Ax and $\mathcal{O}(n)$ extra flops per iteration.

Proof of Conjugate gradient

If $g^k = 0$, then $x^k = x^*$ is solution of the linear system $Ax = b$.

Remark: We set $d^{-1} = 0$ and $\alpha_0 = 0$, so that the recurrence

$d^k = g^k + \alpha_k d^{k-1}$ also covers the case $k = 0$.

For $k = 1$, we have $d^0 = g^0$, so:

$$\begin{aligned} & \langle g^1, d^0 \rangle \\ &= \langle Ax^1 - b, d^0 \rangle \\ &= \langle Ax^0 - b, d^0 \rangle - \beta_0 \langle Ad^0, d^0 \rangle \\ &= \langle g^0, d^0 \rangle - \beta_0 \langle Ad^0, d^0 \rangle \\ &= 0 \end{aligned} \tag{1}$$

by definition of β_0 . This leads to

$$\langle g^1, g^0 \rangle = \langle g^1, d^0 \rangle = 0$$

and

$$\langle d^1, Ad^0 \rangle = \langle g^1, Ad^0 \rangle + \alpha_1 \langle d^0, Ad^0 \rangle = 0$$

by definition of α_1 .

Proof of Conjugate gradient

One can prove the result by recurrence assuming that:

$$\langle g^k, g^j \rangle = 0 \text{ for } 0 \leq j < k$$

$$\langle g^k, d^j \rangle = 0 \text{ for } 0 \leq j < k$$

$$\langle d^k, Ad^j \rangle = 0 \text{ for } 0 \leq j < k$$

If $g^k \neq 0$, the algorithm computes x^{k+1} , g^{k+1} and d^{k+1} .

Proof of Conjugate gradient

- By construction one has $\langle g^{k+1}, d^k \rangle = 0$ (cf. (1)).
- For $j < k$:

$$\begin{aligned} & \langle g^{k+1}, d^j \rangle \\ &= \langle g^{k+1}, d^j \rangle - \langle g^k, d^j \rangle \\ &= \langle g^{k+1} - g^k, d^j \rangle \\ &= -\beta_k \langle Ad^k, d^j \rangle \\ &= 0 \text{ (recurrence hypothesis)} \end{aligned}$$

- For $j \leq k$:

$$\langle g^{k+1}, g^j \rangle = \langle g^{k+1}, d^j \rangle - \alpha_j \langle g^{k+1}, d^{j-1} \rangle = 0 ,$$

since $g^j = d^j - \alpha_j d^{j-1}$.

Proof of Conjugate gradient

- Now: $d^{k+1} = g^{k+1} + \alpha_{k+1}d^k$. For $j < k$

$$\begin{aligned}\langle d^{k+1}, Ad^j \rangle \\ &= \langle g^{k+1}, Ad^j \rangle + \alpha_{k+1} \langle d^k, Ad^j \rangle \\ &= \langle g^{k+1}, Ad^j \rangle .\end{aligned}$$

As $g^{j+1} = g^j - \beta_j Ad^j$, one obtains

$$\langle g^{k+1}, Ad^j \rangle = \frac{1}{\beta_j} \langle g^{k+1}, g^j - g^{j+1} \rangle = 0 \text{ if } \beta_j \neq 0.$$

This implies that if $\beta_j \neq 0$, $\langle d^{k+1}, Ad^j \rangle = 0$ for $j < k$.

- Furthermore one has $\langle d^{k+1}, Ad^k \rangle = 0$.
- So $\langle d^{k+1}, Ad^j \rangle = 0$ for $j < k + 1$.

Proof of Conjugate gradient

- This completes the proof for $\beta_j \neq 0$ and $g^j \neq 0$.
- However one has that

$$\langle g^k, d^k \rangle = \langle g^k, g^k \rangle + \alpha_k \langle g^k, d^{k-1} \rangle = \|g^k\|^2 ,$$

$$\text{and } \beta_k = \frac{\langle g^k, d^k \rangle}{\langle A d^k, d^k \rangle} .$$

- So β_k can only be 0 if $g^k = 0$, which would imply that $x^k = x^*$.
- Furthermore

$$\|d^k\|^2 = \|g^k\|^2 + \alpha_k^2 \|d^{k-1}\|^2 .$$

So if $g^k \neq 0$ then $d^k \neq 0$.

Proof of Conjugate gradient

- Consequently, if the vectors $g^0, g^1, \dots, g^k$ are all non-zero, the vectors $d^0, d^1, \dots, d^k$ are also non-zero.
- The $k + 1$ directions $d^0, d^1, \dots, d^k$ are orthogonal for the dot product $\langle u, v \rangle_A := \langle u, Av \rangle$ (well defined since $A \succ 0$), and the $k + 1$ gradients $g^0, g^1, \dots, g^k$ are orthogonal for the usual dot product $\langle \cdot, \cdot \rangle$.
- Being orthogonal and non-zero, they are linearly independent. In $\mathbb{R}^n$ there can be at most n of them: if $g^0, g^1, \dots, g^{n-1}$ are all non-zero, one necessarily has $d^n = g^n = 0$.
- So it converges after n iterations at the most.



How fast in practice?

“At most n iterations” is of little help when $n = 10^6$. What matters is the accuracy reached after $k \ll n$ iterations.

With $\|u\|_A = \sqrt{\langle u, Au \rangle}$ and $\kappa = \frac{\lambda_{\max}(A)}{\lambda_{\min}(A)}$ the condition number of A :

$$\text{CG: } \|x^k - x^*\|_A \leq 2 \left(\frac{\sqrt{\kappa} - 1}{\sqrt{\kappa} + 1} \right)^k \|x^0 - x^*\|_A$$

$$\text{Gradient descent: } \|x^k - x^*\|_A \leq \left(\frac{\kappa - 1}{\kappa + 1} \right)^k \|x^0 - x^*\|_A$$

→ reaching a precision ε costs $\mathcal{O}(\sqrt{\kappa})$ iterations for CG, against $\mathcal{O}(\kappa)$ for gradient descent.

Remark: Admitted here, cf. Nocedal & Wright, Chapter 5.

$\sqrt{\kappa}$ versus κ

Exercise

Number of iterations to reach a precision 10^{-6} ?

$\sqrt{\kappa}$ versus κ

Exercise

Number of iterations to reach a precision 10^{-6} ?

κ	CG	gradient descent
10^2	≈ 70	≈ 700
10^4	≈ 730	$\approx 69\,000$
10^6	$\approx 7\,300$	$\approx 6\,900\,000$

→ CG is not “a constant factor” better: the gap *widens* as the problem gets worse conditioned.

Remark: These are worst-case bounds. A matrix whose eigenvalues are clustered is solved much faster than $\sqrt{\kappa}$ suggests.

What it means for ridge regression

For $A = X^\top X + \lambda I_p$, with σ_i the singular values of X :

$$\kappa = \frac{\sigma_{\max}^2 + \lambda}{\sigma_{\min}^2 + \lambda}$$

- $\lambda \rightarrow \infty$: $\kappa \rightarrow 1$, the problem becomes trivial.

Remark: This is why a path is computed from the largest λ down.

- $n < p$: $\sigma_{\min} = 0$, so $\kappa = \sigma_{\max}^2 / \lambda$.

Remark: The regularization is what makes the problem solvable, not only well-posed.

→ The regularization parameter controls the *statistics* (bias/variance) and the *cost* of the optimization at the same time.

Getting rid of A : the Fletcher–Reeves form

Exercise

Using $g^k = g^{k-1} - \beta_{k-1} A d^{k-1}$, $\langle g^k, g^{k-1} \rangle = 0$ and $\langle g^k, d^k \rangle = \|g^k\|^2$, show that

$$\alpha_k = -\frac{\langle g^k, A d^{k-1} \rangle}{\langle d^{k-1}, A d^{k-1} \rangle} = \frac{\|g^k\|^2}{\|g^{k-1}\|^2} .$$

Getting rid of A : the Fletcher–Reeves form

Exercise

Using $g^k = g^{k-1} - \beta_{k-1}Ad^{k-1}$, $\langle g^k, g^{k-1} \rangle = 0$ and $\langle g^k, d^k \rangle = \|g^k\|^2$, show that

$$\alpha_k = -\frac{\langle g^k, Ad^{k-1} \rangle}{\langle d^{k-1}, Ad^{k-1} \rangle} = \frac{\|g^k\|^2}{\|g^{k-1}\|^2}.$$

- A has disappeared: the direction coefficient is only a ratio of gradient norms. The same identity gives $\beta_k = \frac{\|g^k\|^2}{\langle d^k, Ad^k \rangle}$.
- Only the *step* β_k still involves A . Replace it by a line search and the recursion applies to *any* smooth f .

Remark: Careful: Nocedal & Wright use α_k for the step size and β_k for the direction coefficient, *i.e.*, the opposite of our notations.

Conjugate gradient for a general smooth f

Require: f smooth, $x^0 \in \mathbb{R}^n$, $\varepsilon > 0$

```
1:  $g^0 = \nabla f(x^0)$ ,  $d^0 = g^0$ 
2: for  $k = 0, 1, 2, \dots$  do
3:   if  $\|g^k\| \leq \varepsilon$  then
4:     return  $x^k$ 
5:   end if
6:   find  $\beta_k$  by line search on  $\beta \mapsto f(x^k - \beta d^k)$ 
7:    $x^{k+1} = x^k - \beta_k d^k$ 
8:    $g^{k+1} = \nabla f(x^{k+1})$ 
9:    $\alpha_{k+1} = \frac{\|g^{k+1}\|^2}{\|g^k\|^2}$  {Fletcher-Reeves}
10:   $d^{k+1} = g^{k+1} + \alpha_{k+1} d^k$ 
11: end for
```

- **Polak-Ribière:** $\alpha_{k+1} = \frac{\langle g^{k+1}, g^{k+1} - g^k \rangle}{\|g^k\|^2}$, identical for a quadratic, more robust otherwise. It restarts on its own: if progress stalls, $g^{k+1} \approx g^k$ gives $\alpha_{k+1} \approx 0$, i.e., a gradient descent step.
- Finite termination is lost: $\nabla^2 f$ is no longer constant.
- This is `scipy.optimize.fmin_cg` (Polak-Ribière).

Note on warm starts and paths

In machine learning it is common to try to solve a problem that is very similar to a previous one.

- You train a model every day and you need just to “update” the model
- You look for the best hyperparameter and evaluate the parameter on a grid of values to get a so-called “path” of solutions. For example on a grid of λ when doing cross-validation.

What it implies for optimization:

- **Warm start:** simply take $x^0 =$ previous solution. Free to implement, and CG only cares about how far x^0 is from x^* .
- It only helps **iterative** solvers. A direct solver pays the same factorization cost whatever the starting point.
- The counterpart for direct solvers on a path: factorize *once* and reuse. With the SVD of X computed once, each new λ is cheap.

Which end of the path do you start from?

Exercise

You need the ridge solutions for a decreasing grid

$$\lambda_1 > \lambda_2 > \cdots > \lambda_T.$$

→ In which order do you solve them, and why?

Which end of the path do you start from?

Exercise

You need the ridge solutions for a decreasing grid

$\lambda_1 > \lambda_2 > \dots > \lambda_T$.

→ In which order do you solve them, and why?

- Start from the **largest** λ : $X^\top X + \lambda I$ is then dominated by λI , hence well conditioned and easy to solve. A crude x^0 (say 0) is already good enough.
- Then decrease λ , each solution warm-starting the next one: consecutive problems differ little, so each solve takes few iterations.
- Going the other way, the first problem solved is the hardest one and you have nothing to start it from.

Remark: Unlike the Lasso, ridge has no finite λ for which $\hat{w} = 0$.

More

Note: Conjugate gradient for sparse linear systems is implemented in `scipy.sparse.linalg.cg`

Note: Conjugate gradient for general smooth problems is implemented in `scipy.optimize.fmin_cg`

Note: `sklearn.linear_model.Ridge` has many solvers. Since v0.18 you have `'svd'`, `'cholesky'`, `'lsqr'`, `'sparse_cg'`, `'sag'` and `'auto'` mode.

→ more in the lecture notes.

→ cf. notebook

References

- Nocedal and Wright, Numerical Optimization, 1999, Springer, Chapter 5.